UNIVERSIDAD DE CARABOBO
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación
Unidad Académica de Algoritmos y Programación
CAO402: Algoritmos II

Profesor: Álvaro Espinoza
Fecha: 29/04/2025

PARCIAL REPARACION

PARTE I: Lea cuidadosamente la pregunta y seleccione la opción correcta. Se aplicará factor de corrección: 3 respuestas incorrectas eliminarán una correcta, una respuesta en blanco no afectará a las respuestas correctas **(4 pts) (0.4 pts c/u)**

a) Sea B una clase que hereda de A y a su vez, una clase C es compuesta por A:
   i) B es contenido de C.
   ii) Sólo A es contenido de C.
   iii) C es clase madre de A.
   iv) Si el parcial fuese sido en diciembre, estuviera más sencillo
   v) C tiene acceso a los métodos privados de B.

b) Se tiene una clase X, con un atributo que es un arreglo de objetos de clase Y. Los objetos de este arreglo se construyen pasando un conjunto de parámetros para su construcción en el constructor de la clase X, esto es un ejemplo de:
   i) Agregación
   ii) Composición
   iii) Polimorfismo
   iv) Friendzone
   v) Clase de asociación

c) Si B es una clase contenido de agregación de A, entonces:
   i) Cuando se crea un objeto de clase B se crea automáticamente un objeto de clase A que es contenido de B.
   ii) El ciclo de vida de A es el mismo que el de B.
   iii) La destrucción de A no implica la destrucción de B.
   iv) B tiene acceso a los atributos privados de A.
   v) Las consecuencias de esta asociación son las mismas que si A fuera clase heredada de B.

d) Dada una clase A con todos sus atributos privados:
   i) Toda clase hija X hereda sus atributos y puede acceder a ellos.
   ii) Una clase Y contenedora de A puede acceder a los atributos de A.
   iii) Debería de haber musica de fondo durante los parciales
   iv) Es posible que un metodo publico de A permite a una clase hija X modificar los atributos heredados de A.
   v) Debería de haber musica de fondo durante los parciales
   vi) Toda clase hija X solo heredará los métodos públicos de A y no sus atributos.

e) Dada una clase A que hereda de B y una clase C que compone a A:
   i) Se puede crear un objeto de clase C utilizando a un objeto de la clase B como componente.
   ii) C tiene acceso a los atributos y métodos privados de B.
   iii) Sólo un objeto de clase A puede servir de contenido para un objeto de clase C.
   iv) Es posible demostrar con métodos avanzados de inducción completa que esta opción no es la correcta.
   v) Ninguna de las demás es la correcta

f) Seleccione la verdadera:
   i) Se puede acceder a un atributo privado de la clase madre en una herencia usando métodos públicos/protegidos setters y getters.
   ii) Mister Satán venció a Cell.
   iii) Una clase hija con todos los métodos de sus ancestros sobrecargados puede acceder a los métodos de su clase abuela.
   iv) Las clases hijas pueden usar atributos privados de los padres directamente.

v) Una clase A con un vector que contiene elementos de clase B SIEMPRE será una composición.

g) Si A es una clase agregada de B, entonces::
   i) Cuando se crea un objeto de clase B se crea automáticamente un objeto de clase A que es contenido de B.
   ii) La destrucción de B no implica la destrucción de A.
   iii) El ciclo de vida de A es el mismo que el de B.
   iv) B tiene acceso a los atributos privados de A.
   v) Las consecuencias de esta asociación son las mismas que si A fuera clase heredada de B.

h) Sea X una clase componente de B, entonces:
   i) La eliminación de X implica la eliminación de B.
   ii) El polimorfismo en X pasaría a ser dinámico.
   iii) La construcción de cada objeto X dentro de B es mediante parámetros y es B quién realiza dicha construcción.
   iv) Al profesor le llamaban "Integrador" cuando era estudiante.
   v) Una clase Y que hereda de X no es componente de B.

i) Si una clase A contiene un atributo x que es un apuntador a un objeto de la misma clase A:
   i) Si B es una clase hija de A, entonces el atributo x ahora será un objeto de tipo B a causa de la herencia.
   ii) x obligatoriamente tendría que ser privado al ser de la misma clase que su contenedor.
   iii) Estoy mareado de tanta clase por aquí y tanta clase por allá.
   iv) Es posible para A acceder a los atributos de $x$desreferenciado en cualquier momento.
   v) Si x es público entonces las clases hijas de A no podrían tener acceso a él porque el atributo es de clase A.

j) Se ejecuta un programa, este falla por una mala implementación de un método sobreescrito, ¿en qué momento debería mostrar el error?:
   i) El error eres tú que lo programaste.
   ii) Cuando se ejecuta el comando para compilar el código.
   iii) No falla.
   iv) En la ejecución del código, justo cuando se invoca el método.
   v) Cuándo la cantidad de datos aumenta enormemente.

Parte II: ejercicios

1. Sea una lista de enteros:

$$A = [a_1, a_2, \ldots, a_n]$$

Definimos una **sublista contigua** como cualquier subsecuencia de la forma

$$[a_i, a_{i+1}, \ldots, a_j]\ para\ 1 \leq i \leq j \leq n$$

El objetivo es encontrar el valor máximo posible de la suma de un subarreglo contiguo, es decir, calcular:

$$\max_{1 \leq i \leq j \leq n} \sum_{k=i}^{j} a_k$$

Se pide que cree la función:

```
func getMaxContiguousSublistIndexes(List<int> target): List<int>
```

Que retorne una lista de dos elementos que son los índices i y j que forman la sublista contigua con mayor suma. NO utilice apuntadores en su solución. Su solución debe ser a lo sumo O(n). **(Valor: 5 puntos)**

2. Dado un árbol n ario (puede tener valores negativos), encuentra el **máximo valor** de la suma de los nodos en **cualquier camino**. El camino puede comenzar y terminar en **cualquier** nodo. Cree el metodo:

```
func ArbolN<int>::getMaxValueSumPath(): int
```

Su solucion debe ser a lo sumo O(n). DEBE USAR APUNTADORES EN SU SOLUCION. **(Valor 6 puntos)**

3. Sea $G = (V, A)$ un grafo no dirigido simple y conexo, donde V es el conjunto de vértices de G, A es es el conjunto de arcos de G. Dados dos vértices $x, y \in V$ $(x \neq y)$, un camino desde x hasta y en G es una secuencia de vértices $C(x, y) = (x = v_0, v_1, \dots, v_k = y)$. En este caso, x y y se denominan extremos de $C(x, y)$, y los vértices restantes se denominan vértices interiores de $C(x, y)$. Si todos los vértices que constituyen $C(x, y)$ son distintos entre si, se dice que $C(x, y)$ es un camino elemental.

   Un enrutamiento R sobre G es un conjunto de caminos elementales que conectan un par (x, y) de vértices distintos en G.

$$R = \{C(x, y): (x, y \in V) \wedge (x \neq y)\}$$

   Entonces, se define la congestión de un arco a con respecto a R, denotada por $\pi(G, R, a)$, es el número de caminos especificados en R que pasan por e. Por lo tanto, se pide que cree la función

```
func congestionArcos(Grafo<int>: g, Lista<entero> xs, ys,
        Lista<Lista<int>> enrutamiento): Lista<entero>
```

   que devuelva una lista de tamaño $|A|$ que devuelva la congestión de cada uno de los arcos de G. Las listas xs y ys son paralelas, tienen tamaño $|A|$ y la posición i de xs está asociada a la posición i de xs, y ambas representan el arco que conecta al vértice xs_i con ys_i. Por lo tanto, la lista resultante debe ser paralela a xs y ys. **(Valor 5 puntos)**